INTERNSHIP PROJECT REPORT

Development and Testing of Custom SIEM Detection Queries

Credential Stuffing, DNS Tunnelling and Suspicious PowerShell Activity Using the ELK Stack

Student name	Abhinav Pratap Singh
Internship organization	Elevance Skills
Internship / project	Cybersecurity Internship — SIEM Correlation Detection

Scope note: This report documents a local educational lab using synthetic security telemetry. No real attacks were performed; records marked simulation=true are not evidence of activity on a production network.

Abstract

This project demonstrates the setup and testing of a small Security Information and Event Management (SIEM) laboratory using Elasticsearch, Logstash and Kibana (the ELK Stack). A synthetic JSON-lines dataset was created to represent benign activity and three security scenarios: repeated failed authentication attempts, unusually long DNS query names, and suspicious PowerShell process command lines. Logstash ingested the events and enriched them with a lab-environment field; Elasticsearch indexed them; and Kibana Discover was used to inspect and filter the resulting records.

The Elasticsearch count API returned 212 indexed documents. Three Elasticsearch detection queries each returned 20 matching documents for their corresponding simulated scenario. Kibana screenshots independently show 20 documents for credential stuffing, 20 for DNS tunnelling, and 20 for suspicious PowerShell within the displayed time range. The work validates ingestion, field availability and scenario-based query matching in a controlled lab. It does not represent detection of live attacks or a production-grade, time-windowed correlation engine.

1. Introduction

A SIEM platform centralizes security telemetry so analysts can search events, identify suspicious patterns and investigate potential incidents. This project uses the ELK Stack as a learning environment to demonstrate a basic security-event pipeline and test detection logic against repeatable synthetic data.

The selected scenarios were credential stuffing, DNS tunnelling and suspicious PowerShell activity. These scenarios were represented as structured event records rather than by performing attacks. This approach allowed repeatable testing without targeting real accounts, infrastructure or systems.

2. Objectives

• Deploy Elasticsearch, Logstash and Kibana locally using Docker Compose.

• Ingest structured JSON security events through Logstash.

• Store and search events in Elasticsearch.

• Build and test query logic for three specified security scenarios.

• Inspect matching and benign events in Kibana Discover.

• Record the observed results and limitations for a technical report.

3. Lab Architecture and Environment

The lab ran on a Windows 11 host with Docker Desktop and WSL2 integration. Docker Compose orchestrated three containers: Elasticsearch 8.19.0, Kibana 8.19.0 and Logstash 8.19.0. Elasticsearch was exposed on host port 9200, Kibana on port 5601, and the Logstash configuration and input data were mounted from the project directory.

Data flow: Synthetic JSON-lines file → Logstash file input and JSON codec → Logstash mutate enrichment → Elasticsearch index (security-events-YYYY.MM.dd) → Kibana Data View (security-events-*) → Discover and Elasticsearch query tests.

Logstash added the field lab_environment=siem-internship-lab. The event records included fields such as @timestamp, event_type, source_ip, scenario and simulation. Scenario-specific fields included authentication_result and user for authentication events, dns_query and dns_query_length for DNS events, and process_name, command_line and suspicious_indicator for process events.

4. Dataset and Test Design

The dataset was generated locally as synthetic telemetry and written to data/security-events.json. The intended dataset design included normal authentication events, credential-stuffing-like failures, ordinary DNS lookups, long DNS query names, benign process events and suspicious PowerShell process telemetry. All generated scenarios were marked simulation=true. No malicious PowerShell command was executed and no real attack was performed.

The generator's intended composition was 210 events: 80 normal authentication, 20 credential-stuffing simulation, 50 normal DNS, 20 DNS-tunnelling simulation, 20 benign process and 20 suspicious PowerShell records. The observed Elasticsearch count was 212, indicating that two additional documents were present in the index, likely from earlier manual testing. The exact provenance of those two additional documents was not separately established.

5. Detection Logic

5.1 Credential stuffing: The query filters authentication events where authentication_result is failed and scenario is credential_stuffing. The synthetic scenario uses repeated failures from source IP 10.10.20.15 across multiple test usernames. The query returned 20 matching documents.

5.2 DNS tunnelling: The query filters DNS events where dns_query_length is greater than or equal to 50 and scenario is dns_tunnelling. The generated test records use long, encoded-looking labels under the reserved example.test domain and source IP 10.10.30.25. The query returned 20 matching documents.

5.3 Suspicious PowerShell: The query filters process events where process_name is powershell.exe, suspicious_indicator is true and scenario is suspicious_powershell. The records contain descriptive command-line strings such as simulated encoded-command and hidden-window indicators. These strings were telemetry only and were not executed. The query returned 20 matching documents.

These are event-filtering detection queries evaluated over indexed records. They are useful for validating the scenario fields and identifying the labelled test events, but they are not full temporal correlation rules: no aggregation threshold across a sliding time window, sequence correlation, alert action or production rule lifecycle was implemented or measured.

6. Implementation and Execution

Elasticsearch and Kibana were started through Docker Compose. The Elasticsearch root endpoint returned HTTP 200 and identified version 8.19.0. Logstash logs showed a successful connection to Elasticsearch, the main pipeline starting, and the file input watcher monitoring the JSON-lines input file. Parsed events were visible in Logstash output.

A Kibana Data View named Security Events was configured over security-events-* using @timestamp as the time field. Kibana Discover then displayed the event histogram and records. KQL filters were used to isolate the credential_stuffing, dns_tunnelling, suspicious_powershell and benign scenarios.

7. Results

Elasticsearch count API: 212 indexed documents; both shards succeeded and no shard failures were reported.

Detection query results: credential stuffing — 20 matching documents; DNS tunnelling — 20 matching documents; suspicious PowerShell — 20 matching documents. These counts were observed in PowerShell query output and supported by Kibana Discover screenshots showing 20 documents for each suspicious scenario.

Kibana Discover all-events view: 190 documents were visible in the screenshot for the selected time range. This is not the same measurement as the Elasticsearch count API's 212 total documents. The screenshots show a selected time interval, and the difference was not fully reconciled during the lab. The report therefore preserves both figures with their respective contexts.

Benign activity view: 149 documents were visible in the screenshot when filtering scenario: benign. This is the displayed result count for that view; it is not used to claim a measured false-positive rate.

Overall, all three scenario-specific query tests returned the expected 20 labelled synthetic records. This demonstrates successful ingestion and query matching for the constructed test cases, not measured accuracy against independently labelled real-world traffic.

8. Discussion

The project demonstrates the main SIEM workflow: collecting structured telemetry, normalizing/enriching it, indexing it, searching it and examining records through a dashboard. Using synthetic data made the test cases repeatable and safe. The Kibana Discover views provide visual evidence of the three scenario groups and benign activity.

The tests are deliberately simple and scenario-labelled. Because the queries include the scenario field, the exercise verifies retrieval of known labelled examples; it does not establish that the same logic can distinguish previously unseen attacks from normal traffic. DNS query length alone can produce false positives, PowerShell command-line indicators can occur in legitimate administration, and failed authentication bursts can arise from user error or misconfigured services.

9. Limitations and Future Improvements

• Implement true time-windowed correlation using Elasticsearch aggregations or Elastic Security threshold rules (for example, a threshold of repeated failed logins from one source within a defined interval).

• Remove reliance on the scenario label in detection logic and evaluate detections using observable fields only.

• Add negative/benign test cases for each rule and calculate precision, recall and false-positive rate using known labels.

• Improve DNS detection by considering query entropy, subdomain uniqueness, query frequency, record type and destination context rather than length alone.

• Improve PowerShell detection with structured process telemetry, parent process, user, host, script-block logging and richer command-line indicators.

• Reconcile the 212 Elasticsearch documents with the 190 documents visible in the Kibana time range and document the exact time filter and index range.

• Add alerting, rule versioning, dashboards and saved searches for a more complete operational workflow.

10. Conclusion

The project successfully deployed a local ELK SIEM lab and ingested synthetic security events into Elasticsearch through Logstash. The Elasticsearch count endpoint reported 212 documents. Three scenario-specific detection queries returned 20 matching records each for credential stuffing, DNS tunnelling and suspicious PowerShell telemetry, and Kibana Discover provided visual evidence of the corresponding event groups.

The outcome is a working educational SIEM ingestion and query-validation prototype. Further work is required before describing the detections as production-grade correlation rules, particularly time-windowed aggregation, testing against unlabelled telemetry and quantitative false-positive evaluation.

Appendix A — Evidence and Detailed Screenshot Descriptions

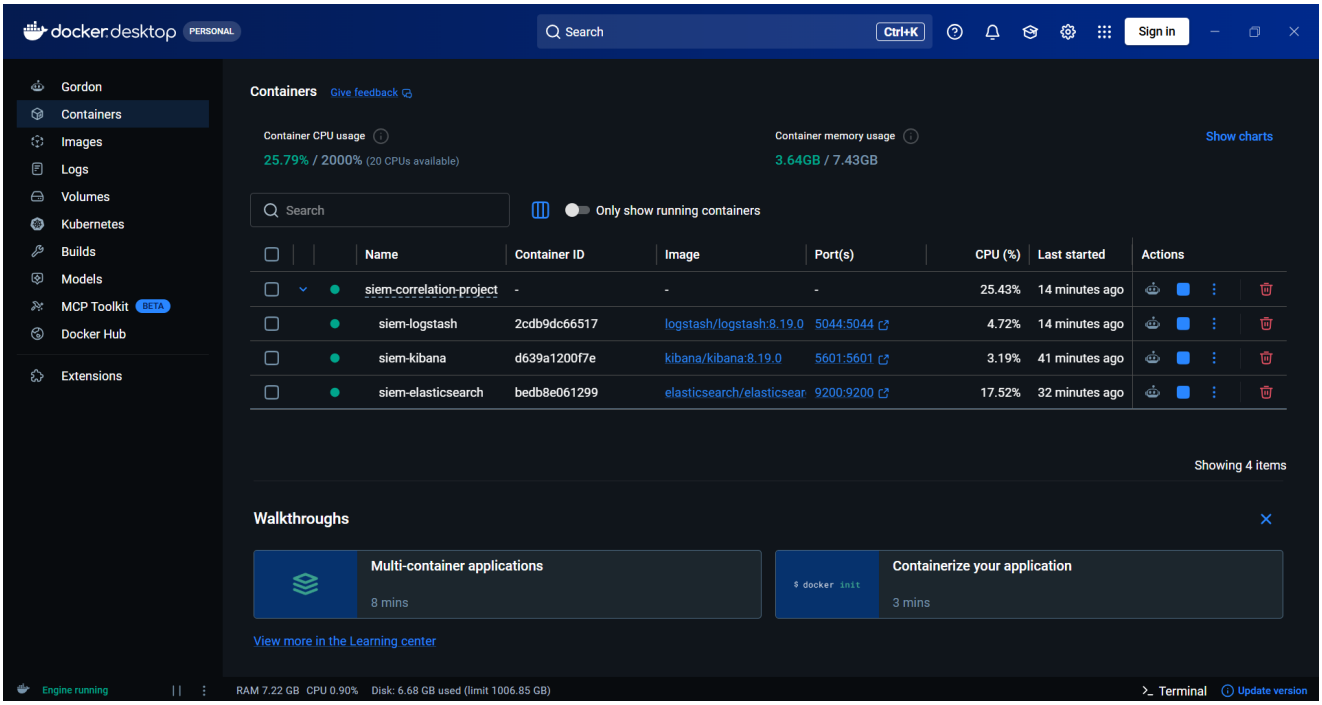


Figure 1. The Docker Desktop view shows the Docker engine running and the Compose application group named siem-correlation-project. Under that group, the siem-logstash, siem-kibana and siem-elasticsearch containers each have a green running indicator. The visible port mappings are 5044:5044 for Logstash, 5601:5601 for Kibana and 9200:9200 for Elasticsearch. This provides visual evidence that the three core services were running together in the local lab at the time of capture.

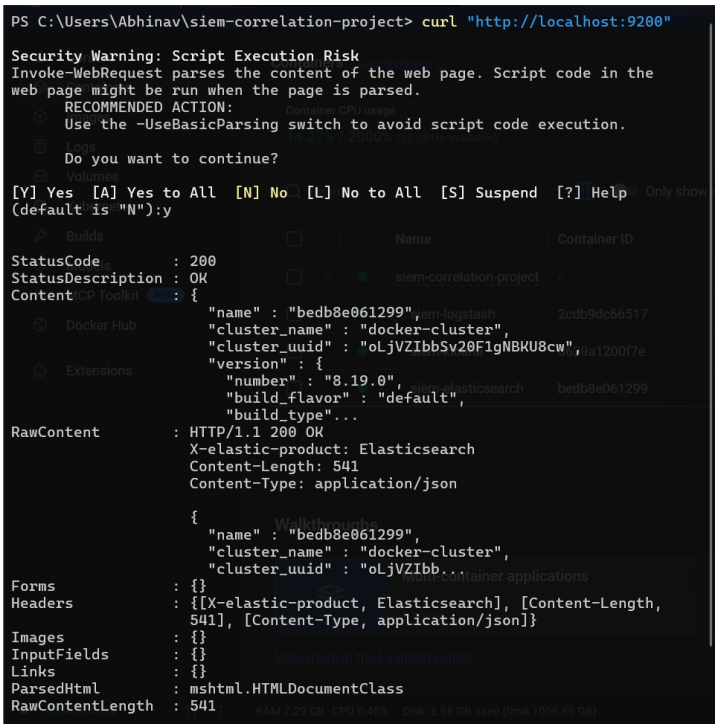


Figure 2. The terminal output shows a successful response from the Elasticsearch root endpoint. The response includes HTTP/1.1 200 OK, identifies Elasticsearch as the product, and displays version 8.19.0. The screenshot also shows the PowerShell curl alias warning because `curl` invokes Invoke-WebRequest; the later command uses curl.exe. The important verification is the successful HTTP 200 response and version information.

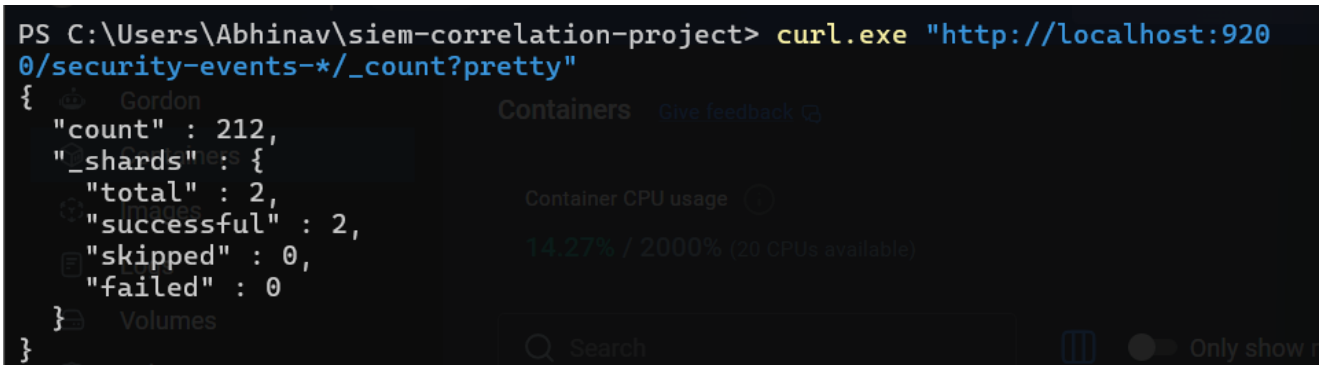


Figure 3. The Elasticsearch `_count` API response reports `count: 212`. The shard summary shows two total shards, two successful, zero skipped and zero failed. This verifies that Elasticsearch returned a successful count for the `security-events-*` index pattern. It is an index-wide count at the time of capture and should not be confused with the number of documents visible in a particular Kibana time filter.

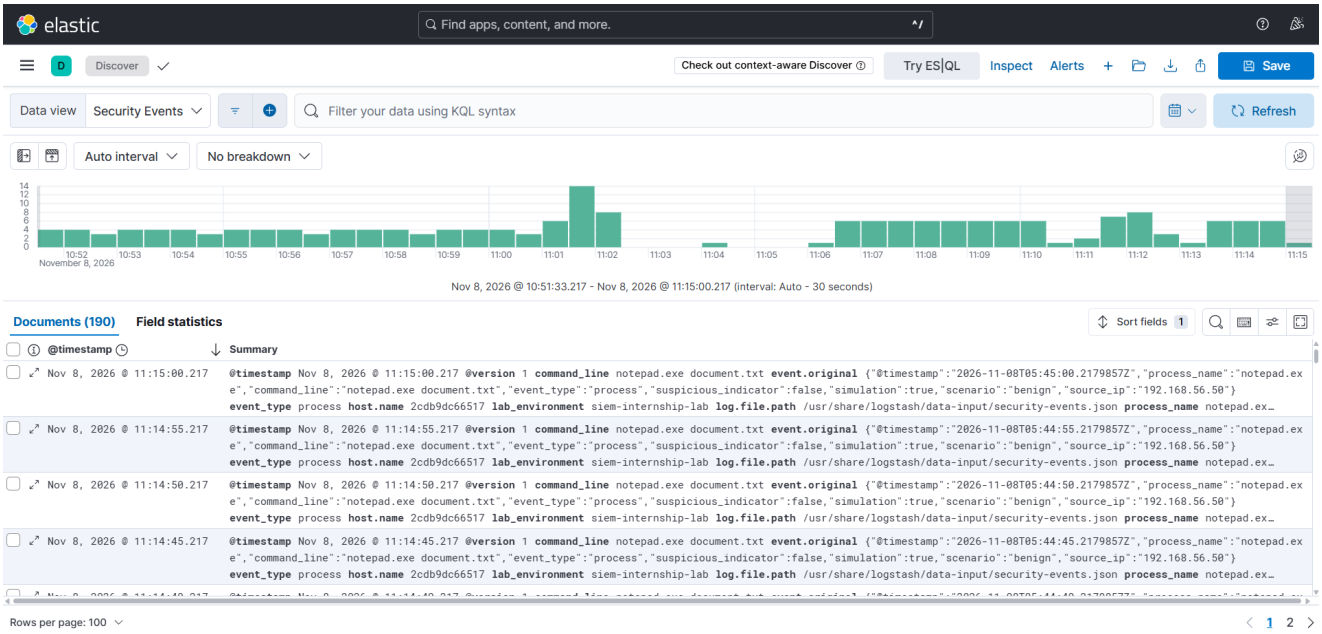


Figure 4. The Discover screen uses the `Security Events` data view and shows a time histogram plus a document table. The displayed table reports 190 documents for the selected time range, with timestamps and event fields visible in the Summary column. This demonstrates that Kibana can query and render the ingested event records. The 190 figure is scoped to the displayed Discover time range, while the Elasticsearch count screenshot reports 212 for the index pattern; the two figures are retained separately because their scopes differ.

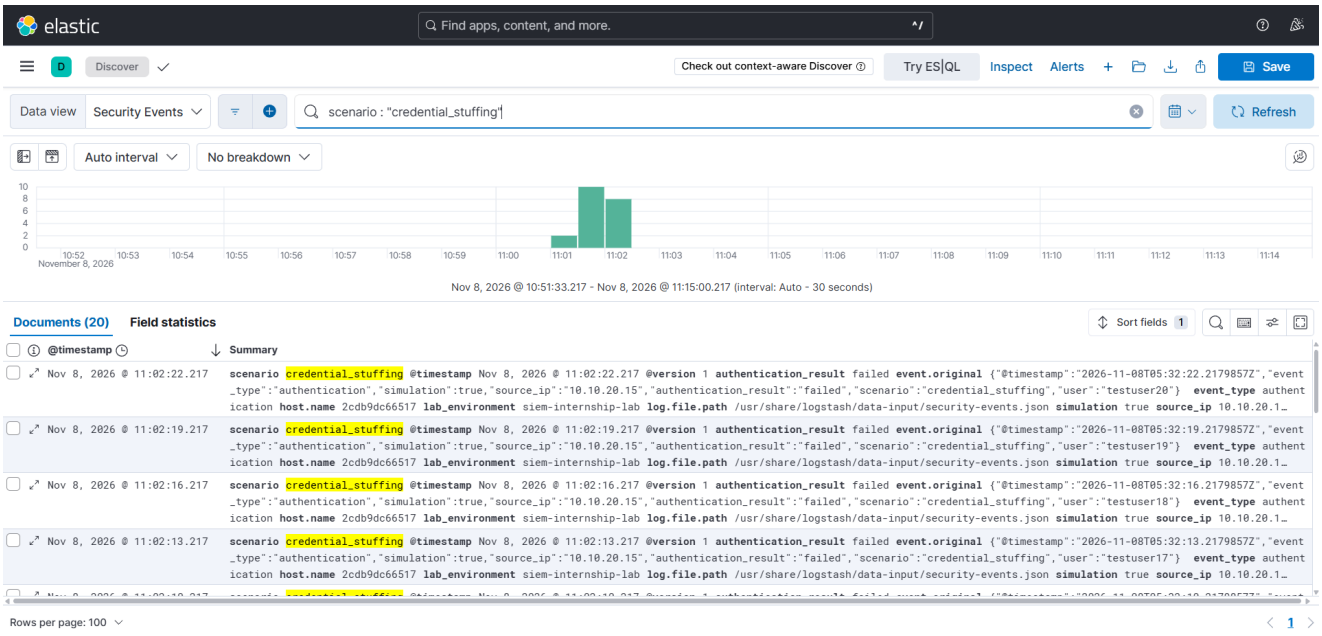


Figure 5. The KQL search bar contains `scenario : "credential_stuffing"`. Discover reports 20 documents. The visible rows show the scenario field, failed authentication result, event type `authentication`, source IP `10.10.20.15`, usernames such as testuser20/testuser19 and timestamps. This is evidence that the credential-stuffing-labelled synthetic events were indexed and can be isolated using the scenario filter.

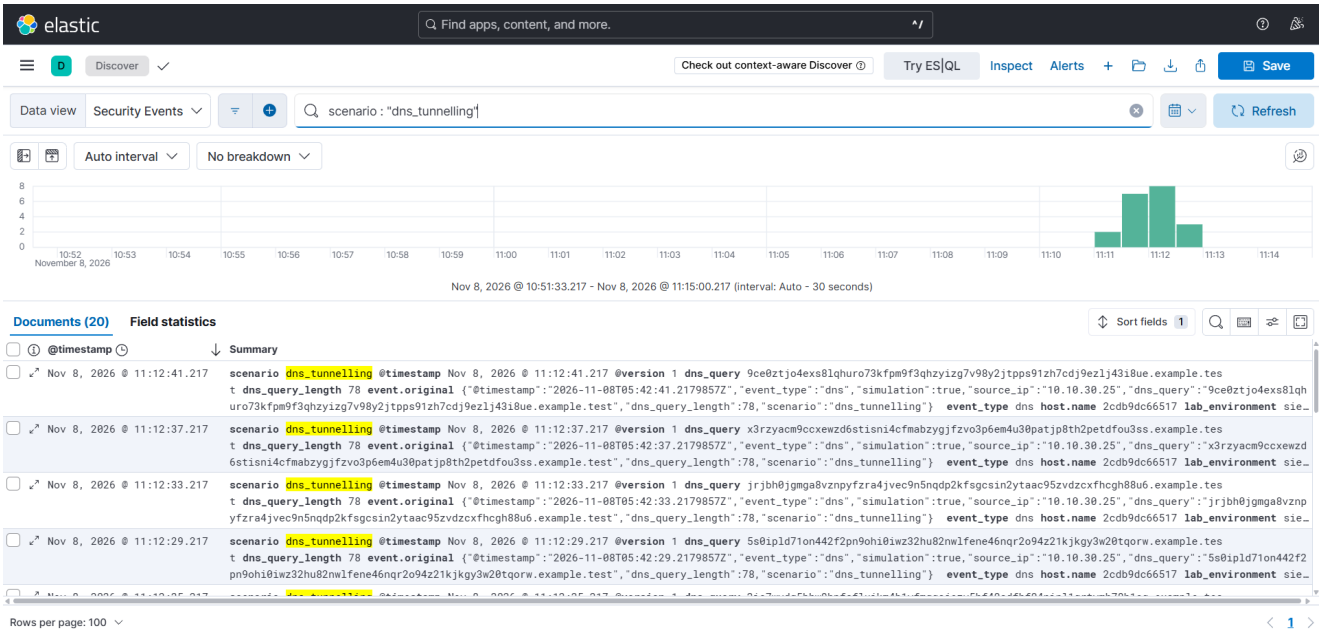


Figure 6. The KQL search bar contains `scenario : "dns_tunnelling"`. Discover reports 20 documents. The visible event rows contain long, encoded-looking DNS query strings under `example.test`, a `dns_query_length` value of 78, event type `dns`, and source IP `10.10.30.25`. This supports the test case that represents unusually long DNS query names. The screenshot does not establish that real DNS tunnelling occurred; these are generated simulation records.

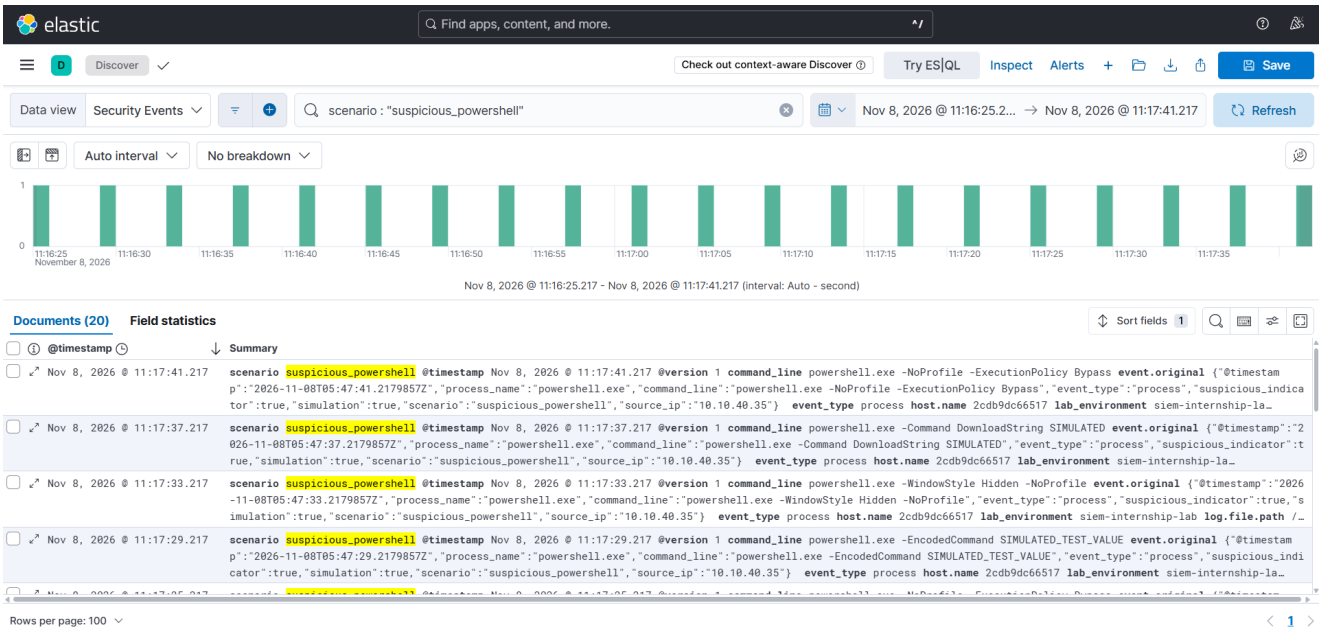


Figure 7. The KQL search bar contains `scenario : "suspicious_powershell"`. Discover reports 20 documents. Visible fields include `process_name` set to `powershell.exe`, command-line strings containing simulated indicators such as `-NoProfile`, `-ExecutionPolicy Bypass`, `-WindowStyle Hidden` and a placeholder encoded-command value, as well as `suspicious_indicator: true` and source IP `10.10.40.35`. These are stored descriptions in synthetic telemetry; the commands were not executed.

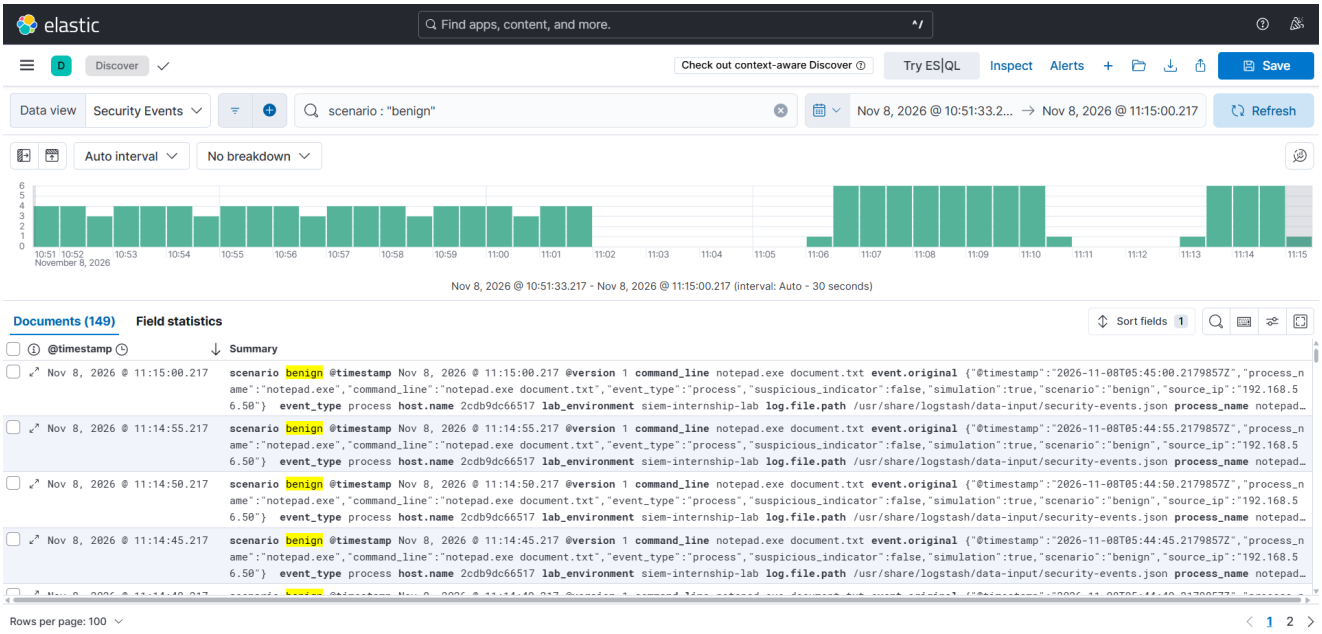


Figure 8. The KQL search bar contains `scenario : "benign"`. Discover reports 149 documents in the displayed result set. Visible rows include ordinary process activity such as `notepad.exe document.txt`, with `suspicious_indicator: false`, and the table also contains benign DNS and authentication records in the dataset. This gives a visual comparison set of normal-labelled events. The screenshot alone is not a formal false-positive measurement.

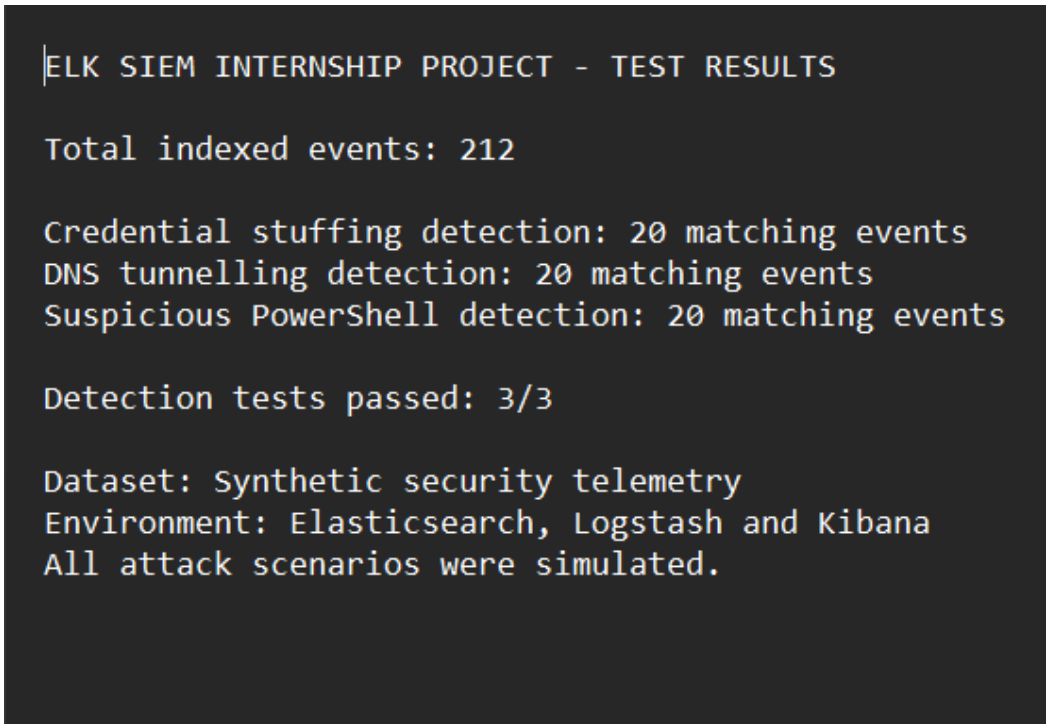


Figure 9. The text summary records the test outcomes: 212 total indexed events; 20 matching events for credential stuffing; 20 for DNS tunnelling; and 20 for suspicious PowerShell; with 3/3 query tests marked passed. It also states that the dataset is synthetic and all attack scenarios were simulated. This is a saved summary of the test run, supported by the individual Kibana and Elasticsearch screenshots. The 3/3 result refers to successful query matches on the constructed scenarios, not production detection accuracy.

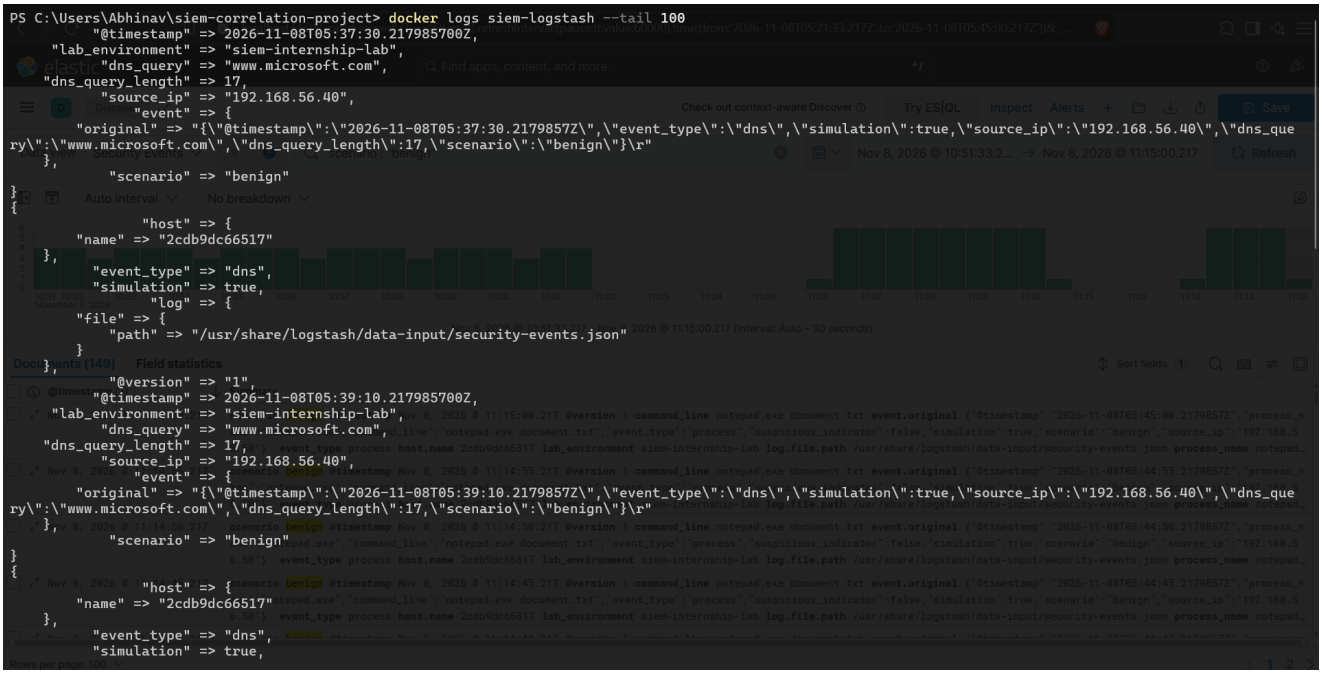


Figure 10. The Logstash terminal output displays parsed event structures emitted by the pipeline. Visible fields include `@timestamp`, `lab_environment`, `dns_query`, `dns_query_length`, `source_ip`, `event_type`, `simulation`, `scenario`, and nested `log.file.path`. The records show Logstash reading from `/usr/share/logstash/data-input/security-events.json`. This demonstrates that the JSON input was parsed into structured events and that the pipeline added the lab-environment metadata.

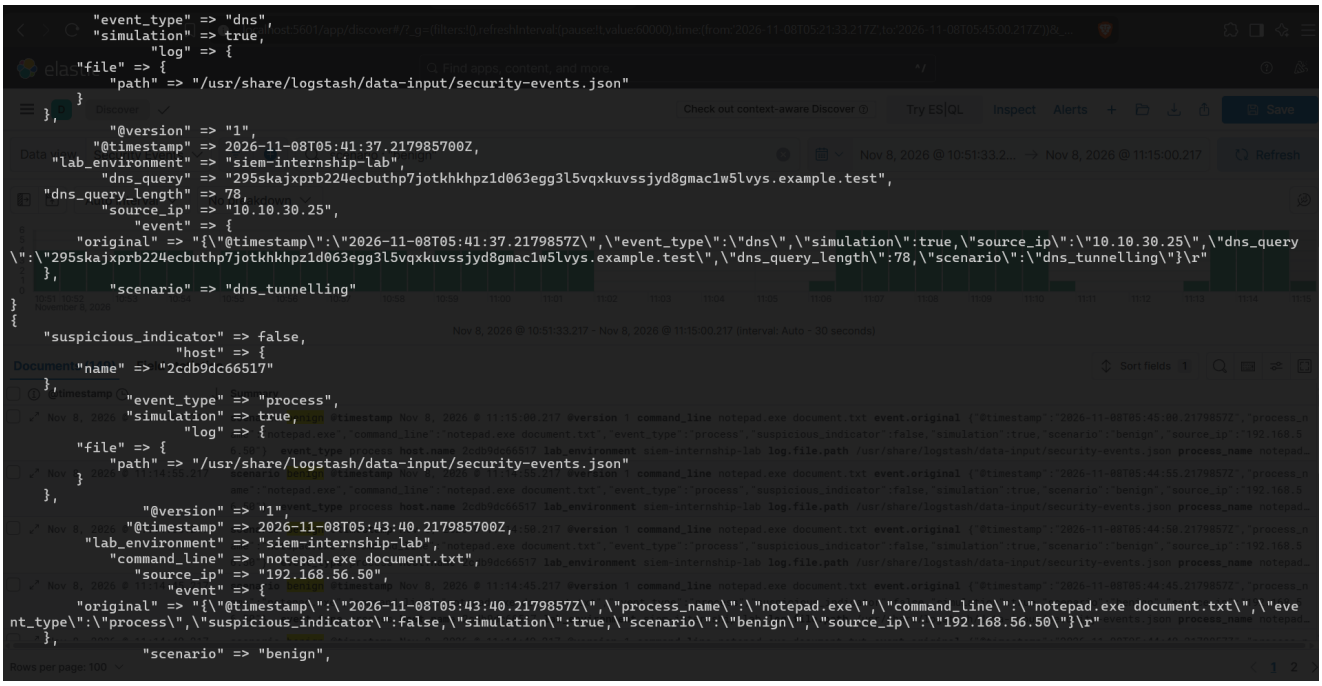


Figure 11. This Logstash output shows a DNS event tagged with `scenario: dns_tunnelling`, a long DNS query ending in `example.test`, `dns_query_length` of 78, and source IP `10.10.30.25`. The nested `event.original` value preserves the original JSON text. This supports the claim that the synthetic DNS scenario was processed by Logstash, rather than merely existing in the source file.



Figure 12. This Logstash output shows a process event with `process_name` equal to `powershell.exe`, a descriptive command line containing `-NoProfile -ExecutionPolicy Bypass`, `suspicious_indicator: true`, `scenario: suspicious_powershell`, and source IP `10.10.40.35`. The event is marked `simulation: true`. It is evidence of ingestion and parsing of simulated PowerShell telemetry only; it is not evidence that PowerShell executed the displayed command.

Appendix B — Reproducibility Notes

Project components: Docker Compose; Elasticsearch 8.19.0; Logstash 8.19.0; Kibana 8.19.0. Input file: data/security-events.json. Logstash pipeline: file input with JSON codec, mutate enrichment, Elasticsearch output to security-events-* daily indices. Kibana Data View: security-events-* with @timestamp.

Suggested validation commands used in the lab included the Elasticsearch root endpoint, the _count API for security-events-*, and scenario-specific _count queries. The three query result counts recorded during execution were 20, 20 and 20 respectively.